

Palindrome Index:-

Code:-

```
import java.io.*;

class Solution {

    public static String longestPalindrome(String s) {

        if (s.length() < 2) {

            return s;

        }

        int start = 0;

        int end = 0;

        for (int i = 0; i < s.length(); i++) {

            int len1 = expand(s, i, i);

            int len2 = expand(s, i, i + 1);

            int len = Math.max(len1, len2);

            if (len > end - start + 1) {

                start = i - (len - 1) / 2;

                end = i + len / 2;

            }

        }

        return s.substring(start, end + 1);

    }

    private static int expand(String s, int left, int right) {

        while (left >= 0 && right < s.length()

            && s.charAt(left) == s.charAt(right)) {

            left--;

            right++;

        }

        return right - left - 1;

    }

    public static void main(String[] args) throws IOException {

        BufferedReader br =

            new BufferedReader(new InputStreamReader(System.in));

        String s = br.readLine();

    }

}
```

```

        System.out.println(longestPalindrome(s));
    }
}

```

Output:-

The screenshot displays the LeetCode submission interface for the 'Longest Palindrome' problem. The submission is 'Accepted' with 144/144 testcases passed. The runtime is 15 ms, which beats 61.90% of other submissions. The memory usage is 43.51 MB, beating 70.62% of other submissions. The code is written in Java and implements a two-pointer approach to find the longest palindromic substring.

Code:

```

21 while (left >= 0 && right < s.length()) {
22     && s.charAt(left) == s.charAt(right)) {
23         left--;
24         right++;
25     }
26     return right - left - 1;
27 }
28 public static void main(String[] args) throws IOException {
29     BufferedReader br =
30         new BufferedReader(new InputStreamReader(System.in));
31     String s = br.readLine();
32     System.out.println(longestPalindrome(s));
33 }
34 }

```

Testcase: Case 1

Input: s = "babad"

Output: "bab"

Circular Palindromes:-

Code:-

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    public static List<Integer> circularPalindromes(String s) {
        List<Integer> result = new ArrayList<>();
        int n = s.length();
        for (int start = 0; start < n; start++) {
            String rotated = s.substring(start) + s.substring(0, start);
            int maxLength = 1;
            for (int i = 0; i < n; i++) {
                int len1 = expand(rotated, i, i);
                int len2 = expand(rotated, i, i + 1);
                maxLength = Math.max(maxLength, Math.max(len1, len2));
            }
            result.add(maxLength);
        }
        return result;
    }

    private static int expand(String s, int left, int right) {
        while (left >= 0 && right < s.length()
            && s.charAt(left) == s.charAt(right)) {
            left--;
        }
    }
}
```

```
        right++;
    }
    return right - left - 1;
}
}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new
FileWriter(System.getenv("OUTPUT_PATH"))));

        int n = Integer.parseInt(bufferedReader.readLine().trim());

        String s = bufferedReader.readLine();

        List<Integer> result = Result.circularPalindromes(s);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}
```

Output:-

✓ **Sample Test case 0**

Input (stdin)

[Download](#)

✓ Sample Test case 1

1	13
2	aaaaabbbbbaaaa

✓ Sample Test case 2

Your Output (stdout)

1	12
2	12
3	10
4	8
5	8
6	9
7	11